Probeklausur

Eine Probeklausur über den Stoff von VL1 bis VL7: 120 Punkte in 120 Minuten, mit Musterlösungen, echten Ausgaben, Punkteschema und Auswertung.

Diese Probeklausur prüft den Stoff von VL1 bis VL7: Schleifen, Datentypen, Listen, Funktionen, Rekursion, Klassen, Vererbung und das Projekt Kaffeeshop. Sie hat 120 Punkte für 120 Minuten. Das ist etwa ein Punkt pro Minute. Alle Aufgaben sind eigene Aufgaben. Sie sind neu und kommen in den Kapiteln 1 bis 8 nicht vor.

So benutzt du die Probeklausur

1. Stell dir einen Timer auf 120 Minuten.
2. Arbeite auf Papier, ohne Rechner, ohne Python und ohne Unterlagen. So übst du den schwierigsten Fall.
3. Schreib bei „Was gibt der Code aus?“ eine Rundentabelle. Sie kostet Zeit, verhindert aber die meisten Fehler.
4. Bleib nicht hängen. Eine Aufgabe, bei der du nach fünf Minuten nicht weiterkommst, überspringst du und kommst später zurück.
5. Erst nach Ablauf der Zeit: Lösungen lesen und Punkte vergeben. Dann tippst du deinen Code ab und lässt ihn laufen, mit `ruff check` und `ty check`.
6. Zum Schluss schaust du unter „Auswertung“ nach, welche Kapitel du wiederholen solltest.

Alle Ausgaben in den Lösungen sind echte Ausgaben. Jedes Programm wurde mit Python ausgeführt, die selbst geschriebenen Funktionen mit `assert` getestet. Die ty-Meldungen stammen von ty 0.0.99. Bei einer anderen Version kann der Wortlaut leicht abweichen.

Aufgabe	Thema	Punkte
1	Was gibt der Code aus? Schleifen	12
2	Was gibt der Code aus? Namen, Listen, Gültigkeit, Operatoren	12
3	Rekursion	12
4	Fehler finden und beheben	12
5	Funktionen schreiben: Strings, Listen, Dictionaries	18
6	Klasse mit Properties und @dataclass	14
7	Vererbung, super(), Polymorphie und ABC	18
8	Typangaben und ty	10
9	Kaffeeshop: Lager und Warenkorb	12
	Summe	**120**

Bestanden mit 60 von 120 Punkten (50 %, Annahme).

Aufgaben

Aufgabe 1 · Schleifen (12 Punkte)

Was geben die Programme aus? Lege zu jedem Programm eine Rundentabelle an.

a) (4 Punkte)

```
summe = 0
for i in range(1, 10, 2):
    if i == 5:
        continue
    summe += i * 2
    print(i, summe)
print("Ende", summe)
```

b) (4 Punkte) Was steht am Ende in `umgedreht` und in `n`?

```
n = 4073
umgedreht = 0
while n > 0:
    ziffer = n % 10
    umgedreht = umgedreht * 10 + ziffer
    n = n // 10
print(umgedreht, n)
```

c) (4 Punkte) Achte auch auf die letzte Zeile.

```
for zeile in range(1, 4):
    werte: list[int] = []
    for spalte in range(1, 5):
        if spalte > zeile + 1:
            break
        werte.append(zeile * spalte)
    print(zeile, werte)
print(zeile, spalte)
```

Aufgabe 2 · Namen, Listen, Gültigkeit, Operatoren (12 Punkte)

a) (4 Punkte) Was wird ausgegeben? Begründe kurz mit „Namen zeigen auf Objekte“.

```
a = [1, 2, 3]
b = a
c = a[:]
b.append(4)
c[0] = 99
a = a + [5]
b[1] = 0
print(a)
print(b)
print(c)
```

b) (3 Punkte) Was wird ausgegeben?

```
rabatt = 10


def preis_mit_rabatt(preis: int) -> int:
    rabatt = 20
    return preis - preis * rabatt // 100


def erhoehe(werte: list[int], betrag: int) -> None:
    for i in range(len(werte)):
        werte[i] += betrag
    betrag = 0
    werte = []


zahlen = [5, 10]
x = 3
print(preis_mit_rabatt(50), rabatt)
erhoehe(zahlen, x)
print(zahlen, x)
```

c) (5 Punkte) Was gibt jede Zeile aus?

```
print(17 // 5, 17 % 5, -17 // 5, 17 / 5)
print("ab" * 3 + str(2 + 3))
print(type(8 / 4).__name__, 0.1 + 0.2 == 0.3)
print(bool(""), bool("0"), bool([]), 3 < 5 < 4)
print(f"{2 / 3:.2f}", round(2.5), round(3.5))
```

Aufgabe 3 · Rekursion (12 Punkte)

a) (4 Punkte) Was gibt `muster(2)` aus?

```
def muster(n: int) -> None:
    if n == 0:
        print("*")
        return
    print("vor", n)
    muster(n - 1)
    print("nach", n)


muster(2)
```

b) (4 Punkte) Gegeben ist diese Funktion:

```
def geheim(n: int) -> str:
    if n < 2:
        return str(n)
    return geheim(n // 2) + str(n % 2)
```

1. Was liefert `geheim(13)`? Schreib die Kette der Aufrufe auf.
2. Was berechnet die Funktion allgemein?
3. Was passiert bei `geheim(13)`, wenn man die beiden Zeilen mit `if n < 2:` und `return str(n)` löscht?

c) (4 Punkte) Schreibe die Funktion `ist_palindrom(text: str) -> bool` **rekursiv**, ohne Schleife und ohne `text[::-1]`. Ein Palindrom liest sich vorwärts und rückwärts gleich. `ist_palindrom("rentner")` ist `True`, `ist_palindrom("kaffee")` ist `False`. Der leere Text und ein einzelnes Zeichen sind Palindrome. Groß- und Kleinschreibung musst du nicht beachten.

Aufgabe 4 · Fehler finden und beheben (12 Punkte)

Die Funktion soll den Durchschnitt aller Noten liefern und zählen, wie viele Noten bestanden sind. Bestanden ist eine Note bis einschließlich 4,0.

```
def auswerten(noten: list[float]) -> tuple[float, int]:
    """Liefert den Durchschnitt und die Anzahl bestandener Noten (bis einschließ
    summe = 0.0
    bestanden = 0
    for i in range(1, len(noten)):
        summe += noten[i]
        if noten[i] < 4.0:
            bestanden + 1
        return summe / len(noten), bestanden


print(auswerten([2.0, 4.0, 5.0]))
```

a) (2 Punkte) Was gibt das Programm so, wie es dasteht, aus? Richtig wäre ungefähr `(3.67, 2)`.

b) (8 Punkte) Im Code stecken vier Fehler. Nenne jeden Fehler mit Zeile und schreib die korrigierte Zeile auf.

c) (2 Punkte) Nach deiner Korrektur: Was passiert bei `auswerten([])`? Ändere die Funktion so, dass sie dann `(0.0, 0)` liefert.

Aufgabe 5 · Funktionen schreiben (18 Punkte)

Schreib jede Funktion mit Typangaben. Diese Hilfen darfst du benutzen:

- `text.split()` zerlegt einen Text an Leerzeichen in eine Liste von Wörtern. Mehrere Leerzeichen hintereinander stören nicht.
- `wort.strip(".,!?")` entfernt diese Zeichen am Anfang und am Ende von `wort`.
- `text.lower()` und `text.upper()` liefern den Text in Klein- bzw. Großbuchstaben.
- Dictionary: `d: dict[str, int] = {}` legt ein leeres an. `d[schluessel] = wert` setzt einen Eintrag, `schluessel in d` prüft, ob er da ist, `d.get(schluessel, 0)` liefert den Wert oder 0. `for schluessel in d:` geht die Schlüssel in Einfügereihenfolge durch.

a) (4 Punkte) `kuerzel(titel: str) -> str` liefert die Anfangsbuchstaben aller Wörter als Großbuchstaben. `kuerzel("Grundlagen der Programmierung")` ist `"GDP"`, `kuerzel("")` ist `""`.

b) (5 Punkte) `woerter_zaehlen(text: str) -> dict[str, int]` zählt, wie oft jedes Wort vorkommt. Groß- und Kleinschreibung zählt nicht, Satzzeichen `.,!?` am Wortrand werden entfernt. `woerter_zaehlen("Kaffee, Tee und Kaffee. KAFFEE!")` ist `{"kaffee": 3, "tee": 1, "und": 1}`.

c) (5 Punkte) `laengste_steigung(werte: list[int]) -> int` liefert die Länge der längsten Folge direkt hintereinander stehender Werte, in der jeder Wert **echt größer** ist als sein Vorgänger. Für `[3, 4, 7, 2, 5, 6, 8, 1]` ist das 4 (2, 5, 6, 8). Für `[5, 5, 5]` ist es 1, für `[]` ist es 0.

d) (4 Punkte) `ueber_durchschnitt(preise: dict[str, float]) -> list[str]` liefert die Namen aller Produkte, deren Preis über dem Durchschnittspreis liegt, in der Reihenfolge des Dictionaries. Ein leeres Dictionary ergibt `[]`. Beispiel: `{"Espresso": 2.2, "Cappuccino": 3.4, "Wasser": 1.5, "Kakao": 3.1}` ergibt `["Cappuccino", "Kakao"]`.

Aufgabe 6 · Klasse mit Properties und @dataclass (14 Punkte)

a) (8 Punkte) Schreibe eine Klasse `Bohnenbehaelter` für eine Kaffeemaschine. Eine Tasse braucht 8 g Bohnen. Lege dafür eine Konstante `GRAMM_PRO_TASSE` mit `Final` an.

- `__init__(self, kapazitaet_g: int)`: Der Behälter ist am Anfang leer.

- Property `kapazitaet_g`: nur lesbar.
- Property `fuellstand_g` mit Setter: Erlaubt sind Werte von 0 bis zur Kapazität. Ein ungültiger Wert gibt `Ungültiger Füllstand: <wert> g` aus, der alte Wert bleibt.
- Berechnete Property `tassen`: Wie viele ganze Tassen reichen die Bohnen?
- Methode `entnehmen(self, tassen: int) -> bool`: nimmt die Bohnen für so viele Tassen heraus und liefert `True`. Ist `tassen` kleiner als 1 oder reichen die Bohnen nicht, ändert sich nichts und die Methode liefert `False`.

b) (3 Punkte) Was gibt dieser Code aus?

```
from dataclasses import dataclass


@dataclass
class Bruehung:
    sorte: str
    tassen: int
    stark: bool = False


erste = Bruehung("Espresso", 2)
zweite = Bruehung("Espresso", 2)
print(erste)
print(erste == zweite, erste is zweite)
print(Bruehung("Lungo", 1, True).stark)
```

c) (3 Punkte) Mit deiner Klasse aus a): `b = Bohnenbehaelter(250)`. Was meldet ty, und was passiert beim Ausführen, (1) bei `b.tassen = 10` und (2) bei `b.fuellstand_g = 500`?

Aufgabe 7 · Vererbung, super(), Polymorphie und ABC (18 Punkte)

a) (6 Punkte) Was gibt das Programm aus?

```
from typing import override


class Getraenk:
    def __init__(self, name: str, preis: int) -> None:
        self.name = name
        self.preis = preis

    def beschreibung(self) -> str:
        return f"{self.name} ({self.preis_text()})"

    def preis_text(self) -> str:
        return f"{self.preis} ct"


class Heissgetraenk(Getraenk):
    def __init__(self, name: str, preis: int, grad: int) -> None:
        super().__init__(name, preis + 20)
        self.grad = grad

    @override
    def preis_text(self) -> str:
        return super().preis_text() + f", {self.grad} °C"


class Kakao(Heissgetraenk):
    @override
    def beschreibung(self) -> str:
        return "Süß: " + super().beschreibung()


karte: list[Getraenk] = [
    Getraenk("Wasser", 150),
    Heissgetraenk("Tee", 200, 80),
    Kakao("Kakao", 250, 65),
]
for g in karte:
    print(g.beschreibung())
print(isinstance(karte[2], Heissgetraenk), type(karte[2]).__name__)
print(isinstance(karte[1], Kakao), karte[2].preis)
```

b) (10 Punkte) Der Kaffeeshop bietet verschiedene Versandarten an. Rechne mit `Decimal`.

- Abstrakte Klasse `Versandart` mit Attribut `name`, der abstrakten Methode `kosten(self, gewicht_g: int) -> Decimal` und der konkreten Methode `angebot(self, gewicht_g: int) -> str`. Sie liefert zum Beispiel `"Standard: 6.99 Euro"`.

- `Standard`: bis einschließlich 2000 g kostet der Versand 4.99 Euro, darüber 6.99 Euro.
- `Express` ist eine Unterklasse von `Standard`. Sie kostet 5.00 Euro mehr als Standard. Rufe dafür die Methode der Oberklasse auf, statt die Preise zu wiederholen.
- `Abholung(filiale)`: kostet 0.00 Euro. Der Name ist `"Abholung in <filiale>"`.
- Funktion `guenstigste(arten: list[Versandart], gewicht_g: int) -> Versandart`: liefert die günstigste Versandart. Die Liste ist nie leer. Die Funktion darf die Klassen nicht unterscheiden (kein `isinstance`).

c) (2 Punkte) Jemand schreibt `class Brief(Versandart)` mit einem `__init__` und einer Methode `porto()`, aber ohne `kosten()`. Das Programm gibt zuerst `Start` aus und führt dann `b = Brief()` aus. Was passiert, und wann? Was meldet ty?

Aufgabe 8 · Typangaben und ty (10 Punkte)

```
from typing import Final

MWST: Final = 0.19


def brutto(netto: float) -> float:
    return round(netto * (1 + MWST), 2)


def groesse(menge: int) -> str:
    if menge > 10:
        return "groß"
    elif menge > 0:
        return "klein"


anzahl: int = 3
anzahl = "drei"  # (1)
MWST = 0.07  # (2)
print(groesse(0))  # (3)
print(brutto(10))  # (4)
print(brutto("10"))  # (5)
print("fertig")
```

a) (5 Punkte) `ty check` meldet genau vier Fehler. Welche Stellen sind es, und was bemängelt ty jeweils? Gehe die Zeilen (1) bis (5) durch und sag zu jeder, ob ty an dieser Stelle etwas meldet.

b) (3 Punkte) Das Programm wird trotzdem mit `python` gestartet. Was wird ausgegeben?

c) (2 Punkte) Korrigiere `groesse()` so, dass ty nichts mehr meldet und `groesse(0)` den Text `"keine"` liefert.

Aufgabe 9 · Kaffeeshop: Lager und Warenkorb (12 Punkte)

Gegeben ist eine vereinfachte Fassung des Kaffeeshops. `Produkt` ist hier eine `@dataclass` und nicht die Klasse aus dem Startpaket.

```
from dataclasses import dataclass
from decimal import Decimal


@dataclass
class Produkt:
    nummer: int
    name: str
    preis: Decimal
    bestand: int


@dataclass
class Position:
    produkt: Produkt
    menge: int


class Lager:
    def __init__(self, produkte: list[Produkt]) -> None:
        self.produkte = produkte

    def finden(self, nummer: int) -> Produkt | None:
        for produkt in self.produkte:
            if produkt.nummer == nummer:
                return produkt
        return None


class Warenkorb:
    def __init__(self) -> None:
        self.positionen: list[Position] = []

    def position_zu(self, nummer: int) -> Position | None:
        for position in self.positionen:
            if position.produkt.nummer == nummer:
                return position
        return None

    def hinzufuegen(self, produkt: Produkt, menge: int) -> bool:
        position = self.position_zu(produkt.nummer)
        im_korb = 0 if position is None else position.menge
        if im_korb + menge > produkt.bestand:
            return False
        if position is None:
            self.positionen.append(Position(produkt, menge))
        else:
            position.menge += menge
```

```
        return True

    def bestellen(self) -> Decimal:
        summe = Decimal(0)
        for position in self.positionen:
            position.produkt.bestand -= position.menge
            summe += position.produkt.preis * position.menge
        self.positionen = []
        return summe
```

a) (3 Punkte) Ergänze `Lager` um die Methode `nachbestellen(self, mindestbestand: int) -> list[Produkt]`. Sie liefert alle Produkte, deren Bestand **unter** `mindestbestand` liegt.

b) (5 Punkte) Ergänze `Warenkorb` um die Methode `reduzieren(self, nummer: int, menge: int) -> bool`. Sie verringert die Menge der Position mit dieser Artikelnummer. Fällt die Menge auf 0 oder darunter, wird die Position ganz entfernt. Liegt das Produkt nicht im Korb oder ist `menge` kleiner als 1, ändert sich nichts und die Methode liefert `False`. Sonst liefert sie `True`.

c) (4 Punkte) Was gibt dieses Programm aus (3 Punkte)? Bei welcher Zeile meldet ty einen Fehler, und warum (1 Punkt)?

```
kaffee = Produkt(101, "Kaffee 500 g", Decimal("8.90"), 5)
becher = Produkt(204, "Thermobecher", Decimal("14.50"), 2)
lager = Lager([kaffee, becher])
korb = Warenkorb()
print(korb.hinzufuegen(kaffee, 3))
print(korb.hinzufuegen(lager.finden(101), 3))
print(korb.hinzufuegen(becher, 2))
gemerkt = korb.positionen
print(korb.bestellen())
print(kaffee.bestand, becher.bestand, len(korb.positionen), len(gemerkt))
print(korb.hinzufuegen(becher, 1))
```

Lösungen und Bewertung

Aufgabe 1

a) `range(1, 10, 2)` liefert 1, 3, 5, 7, 9. Bei 5 springt `continue` sofort in die nächste Runde, ohne zu addieren und ohne `print`.

Runde	i	i == 5?	summe danach	Ausgabe
Start	–	–	0	–
1	1	nein	0 + 2 = 2	`1 2`
2	3	nein	2 + 6 = 8	`3 8`
3	5	ja, `continue`	8	–
4	7	nein	8 + 14 = 22	`7 22`
5	9	nein	22 + 18 = 40	`9 40`

```
1 2
3 8
7 22
9 40
Ende 40
```

Bewertung: je richtige Zeile 1 Punkt, höchstens 4. Wer bei i = 5 etwas ausgibt, verliert 1 Punkt.

b) `n % 10` liefert die letzte Ziffer, `n // 10` streicht sie. `umgedreht * 10 + ziffer` hängt die Ziffer hinten an.

Runde	n vorher	ziffer	umgedreht danach	n danach
Start	–	–	0	4073
1	4073	3	0 · 10 + 3 = 3	407
2	407	7	3 · 10 + 7 = 37	40
3	40	0	37 · 10 + 0 = 370	4
4	4	4	370 · 10 + 4 = 3704	0

Danach ist `n > 0` falsch, die Schleife endet.

```
3704 0
```

Bewertung: 2 Punkte für 3704, 1 Punkt für die 0, 1 Punkt für eine nachvollziehbare Tabelle.

c) Die innere Schleife bricht ab, sobald `spalte > zeile + 1` ist.

zeile	spalte	spalte > zeile + 1?	werte danach
1	1	1 > 2 nein	[1]
1	2	2 > 2 nein	[1, 2]
1	3	3 > 2 ja, `break`	[1, 2]
2	1, 2, 3	nein	[2, 4, 6]
2	4	4 > 3 ja, `break`	[2, 4, 6]
3	1, 2, 3, 4	4 > 4 nein	[3, 6, 9, 12]

In Zeile 3 läuft die innere Schleife ohne `break` bis zum Ende. `spalte` behält danach den letzten Wert 4. `zeile` und `spalte` gibt es auch nach den Schleifen noch, weil `for` keinen eigenen Gültigkeitsbereich hat.

```
1 [1, 2]
2 [2, 4, 6]
3 [3, 6, 9, 12]
3 4
```

Bewertung: je Zeile 1 Punkt. Häufiger Fehler in der letzten Zeile: `3 5` (range(1, 5) endet bei 4) oder ein `NameError`.

Aufgabe 2

a)

```
[1, 2, 3, 4, 5]
[1, 0, 3, 4]
[99, 2, 3]
```

1. `b = a`: zwei Namen, eine Liste.
2. `c = a[:]`: ein Ausschnitt über die ganze Liste, also eine **Kopie**.
3. `b.append(4)` verändert die gemeinsame Liste: `[1, 2, 3, 4]`.
4. `c[0] = 99` ändert nur die Kopie: `[99, 2, 3]`.
5. `a = a + [5]` erzeugt eine **neue** Liste `[1, 2, 3, 4, 5]` und bindet nur `a` neu. `b` zeigt weiter auf die alte Liste.
6. `b[1] = 0` ändert die alte Liste: `[1, 0, 3, 4]`. `a` merkt davon nichts mehr.

Bewertung: je Zeile 1 Punkt, 1 Punkt für die Begründung (Kopie durch `[:]`, Neubindung durch `a + [5]`).

b)

```
40 10
[8, 13] 3
```

`rabatt = 20` in der Funktion ist eine **lokale** Variable: 50 − 50 · 20 // 100 = 50 − 10 = 40. Die globale `rabatt` bleibt 10. `erhoehe()` verändert die Liste selbst (`werte[i] += betrag`), das ist außen sichtbar. `betrag = 0` und `werte = []` binden nur die lokalen Namen neu. `x` und `zahlen` außen ändern sich dadurch nicht.

Bewertung: 1 Punkt für `40 10`, 1 Punkt für `[8, 13]`, 1 Punkt für `3`.

c)

```
3 2 -4 3.4
abababb5
float False
False True False False
0.67 2 4
```

- `-17 // 5` rundet ab, zur kleineren Zahl: −3,4 wird zu −4.
- `8 / 4` ist immer ein `float` (2.0), auch wenn die Division aufgeht.
- `0.1 + 0.2` ist wegen der binären Speicherung nicht genau 0.3.
- `bool("0")` ist `True`: Der Text ist nicht leer. `3 < 5 < 4` bedeutet `3 < 5 and 5 < 4`.
- `round()` rundet bei ,5 zur geraden Zahl: `round(2.5)` ist 2, `round(3.5)` ist 4.

Bewertung: je Zeile 1 Punkt, nur wenn die ganze Zeile stimmt.

Aufgabe 3

a)

```
vor 2
vor 1
*
nach 1
nach 2
```

`muster(2)` gibt „vor 2“ aus und wartet auf `muster(1)`. Das gibt „vor 1“ aus und wartet auf `muster(0)`. Das ist der Basisfall: „*“. Danach machen die wartenden Aufrufe weiter, der innerste zuerst: „nach 1“, dann „nach 2“.

Bewertung: 1 Punkt für die beiden „vor“-Zeilen, 1 Punkt für „*“, 2 Punkte für die „nach“-Zeilen in der richtigen Reihenfolge.

b)

1. Aufrufkette:
   - `geheim(13)` = `geheim(6)` + "1"
   - `geheim(6)` = `geheim(3)` + "0"
   - `geheim(3)` = `geheim(1)` + "1"
   - `geheim(1)` = "1" (Basisfall)
   - zusammengesetzt: "1" + "1" + "0" + "1" = `"1101"`
2. Die Funktion liefert die **Binärdarstellung** von n als Text: 13 = 8 + 4 + 1 = 1101_2.
3. Ohne Basisfall ruft sich die Funktion endlos selbst auf: 13, 6, 3, 1, 0, 0, 0 … (0 // 2 ist wieder 0). Python bricht ab:

```
RecursionError: maximum recursion depth exceeded
```

Bewertung: 2 Punkte für `"1101"` mit Kette, 1 Punkt für „Binärzahl“, 1 Punkt für `RecursionError`.

c)

```
def ist_palindrom(text: str) -> bool:
    if len(text) <= 1:
        return True
    if text[0] != text[-1]:
        return False
    return ist_palindrom(text[1:-1])


assert ist_palindrom("rentner")
assert ist_palindrom("abba")
assert ist_palindrom("")
assert ist_palindrom("x")
assert not ist_palindrom("kaffee")
assert not ist_palindrom("ab")
print("alle Tests bestanden")
```

```
alle Tests bestanden
```

`text[1:-1]` ist der Text ohne erstes und letztes Zeichen, also ein kleineres Problem.

Bewertung: 1 Punkt Basisfall (`<= 1` deckt auch den leeren Text ab), 1 Punkt Vergleich erstes und letztes Zeichen, 1 Punkt rekursiver Aufruf mit kleinerem Text und `return`, 1 Punkt Typangaben und richtige Rückgabewerte. Wer nur `len(text) == 1` als Basisfall hat, verliert 1 Punkt: Bei geraden Längen wie „abba“ kommt man beim leeren Text an.

Aufgabe 4

a)

```
(1.3333333333333333, 0)
```

Die Schleife beginnt bei i = 1, also bei 4.0. `summe` wird 4.0. `4.0 < 4.0` ist falsch. Dann kommt schon in der ersten Runde das `return`: 4.0 / 3 = 1.333…, bestanden 0.

Bewertung: 2 Punkte für die richtige Ausgabe, 1 Punkt, wenn nur einer der beiden Werte stimmt.

b)

Nr	Fehler	Korrektur
1	`range(1, len(noten))` lässt die erste Note aus	`for i in range(len(noten)):`
2	`< 4.0` zählt 4,0 nicht als bestanden	`if noten[i] <= 4.0:`
3	`bestanden + 1` rechnet, speichert aber nichts	`bestanden += 1`
4	`return` steht in der Schleife, die Funktion endet nach Runde 1	`return` eine Stufe weniger einrücken, auf Höhe von `for`

Bewertung: je Fehler 1 Punkt fürs Finden und 1 Punkt für die Korrektur.

c) Mit leerer Liste läuft die Schleife nicht. Dann teilt `summe / len(noten)` durch 0:

```
ZeroDivisionError: float division by zero
```

Korrigierte Funktion mit Tests:

```
def auswerten(noten: list[float]) -> tuple[float, int]:
    """Liefert den Durchschnitt und die Anzahl bestandener Noten (bis einschließ
    if len(noten) == 0:
        return 0.0, 0
    summe = 0.0
    bestanden = 0
    for i in range(len(noten)):
        summe += noten[i]
        if noten[i] <= 4.0:
            bestanden += 1
    return summe / len(noten), bestanden


print(auswerten([2.0, 4.0, 5.0]))
assert auswerten([2.0, 4.0, 5.0]) == (11 / 3, 2)
assert auswerten([1.0, 3.0]) == (2.0, 2)
assert auswerten([5.0]) == (5.0, 0)
assert auswerten([]) == (0.0, 0)
print("alle Tests bestanden")
```

```
(3.6666666666666665, 2)
alle Tests bestanden
```

Bewertung: 1 Punkt für `ZeroDivisionError`, 1 Punkt für die Prüfung vor der Division.

Aufgabe 5

```
def kuerzel(titel: str) -> str:
    ergebnis = ""
    for wort in titel.split():
        ergebnis += wort[0].upper()
    return ergebnis


def woerter_zaehlen(text: str) -> dict[str, int]:
    anzahl: dict[str, int] = {}
    for wort in text.lower().split():
        wort = wort.strip(".,!?")
        if wort == "":
            continue  # nur Satzzeichen, z. B. ein einzelnes "!"
        anzahl[wort] = anzahl.get(wort, 0) + 1
    return anzahl


def laengste_steigung(werte: list[int]) -> int:
    if len(werte) == 0:
        return 0
    laengste = 1
    aktuell = 1
    for i in range(1, len(werte)):
        if werte[i] > werte[i - 1]:
            aktuell += 1
            if aktuell > laengste:
                laengste = aktuell
        else:
            aktuell = 1
    return laengste


def ueber_durchschnitt(preise: dict[str, float]) -> list[str]:
    if len(preise) == 0:
        return []
    summe = 0.0
    for name in preise:
        summe += preise[name]
    schnitt = summe / len(preise)
    treffer: list[str] = []
    for name in preise:
        if preise[name] > schnitt:
            treffer.append(name)
    return treffer


assert kuerzel("Grundlagen der Programmierung") == "GDP"
assert kuerzel("  vertrags-  und schuldrecht ") == "VUS"
```

```
assert kuerzel("") == ""
print(woerter_zaehlen("Kaffee, Tee und Kaffee. KAFFEE!"))
assert woerter_zaehlen("Kaffee, Tee und Kaffee. KAFFEE!") == {"kaffee": 3, "tee"
assert woerter_zaehlen("") == {}
assert woerter_zaehlen("Ja ! ja") == {"ja": 2}
print(laengste_steigung([3, 4, 7, 2, 5, 6, 8, 1]))
assert laengste_steigung([3, 4, 7, 2, 5, 6, 8, 1]) == 4
assert laengste_steigung([5, 5, 5]) == 1
assert laengste_steigung([]) == 0
assert laengste_steigung([9]) == 1
assert laengste_steigung([1, 2, 3]) == 3
karte = {"Espresso": 2.2, "Cappuccino": 3.4, "Wasser": 1.5, "Kakao": 3.1}
print(ueber_durchschnitt(karte))
assert ueber_durchschnitt(karte) == ["Cappuccino", "Kakao"]
assert ueber_durchschnitt({}) == []
assert ueber_durchschnitt({"A": 2.0, "B": 2.0}) == []
print("alle Tests bestanden")
```

```
{'kaffee': 3, 'tee': 1, 'und': 1}
4
['Cappuccino', 'Kakao']
alle Tests bestanden
```

Rundentabelle für `laengste_steigung([3, 4, 7, 2, 5, 6, 8, 1])`:

i	werte[i − 1] → werte[i]	größer?	aktuell	laengste
Start	–	–	1	1
1	3 → 4	ja	2	2
2	4 → 7	ja	3	3
3	7 → 2	nein	1	3
4	2 → 5	ja	2	3
5	5 → 6	ja	3	3
6	6 → 8	ja	4	4
7	8 → 1	nein	1	4

Der Durchschnitt in d) ist (2.2 + 3.4 + 1.5 + 3.1) / 4 = 2.55. Darüber liegen Cappuccino und Kakao.

ty meldet für diese Lösung nichts. Sind in ruff die Pylint-Regeln (`PL`) eingeschaltet, schlägt es zwei Kürzungen vor: `laengste = max(laengste, aktuell)` statt des inneren `if` und `for name, preis in preise.items():` statt `preise[name]`. Beides ist richtig, die Fassung oben aber auch.

Bewertung:

- **a) 4 Punkte:** 1 Schleife über `split()`, 1 erstes Zeichen mit `[0]`, 1 `upper()`, 1 Aufbau und Rückgabe des Ergebnisses.
- **b) 5 Punkte:** 1 `lower()` und `split()`, 1 `strip()`, 2 Zählen mit `get()` oder mit `if wort in anzahl: … else: …`, 1 Rückgabe und Typangabe. Das Überspringen leerer Wörter ist ein Extra ohne Punktabzug.
- **c) 5 Punkte:** 1 leere Liste, 1 Vergleich mit dem Vorgänger ohne Indexfehler (Start bei 1), 1 Zähler hochzählen bzw. auf 1 zurücksetzen, 1 Maximum merken, 1 richtiges Ergebnis bei `[5, 5, 5]`.
- **d) 4 Punkte:** 1 Durchschnitt, 1 leeres Dictionary, 1 Vergleich mit `>` (nicht `>=`), 1 Reihenfolge und Rückgabe als Liste.

Aufgabe 6

a)

```
from typing import Final

GRAMM_PRO_TASSE: Final = 8


class Bohnenbehaelter:
    def __init__(self, kapazitaet_g: int) -> None:
        self._kapazitaet_g = kapazitaet_g
        self._fuellstand_g = 0

    @property
    def kapazitaet_g(self) -> int:
        return self._kapazitaet_g

    @property
    def fuellstand_g(self) -> int:
        return self._fuellstand_g

    @fuellstand_g.setter
    def fuellstand_g(self, wert: int) -> None:
        if wert < 0 or wert > self._kapazitaet_g:
            print(f"Ungültiger Füllstand: {wert} g")
            return
        self._fuellstand_g = wert

    @property
    def tassen(self) -> int:
        return self._fuellstand_g // GRAMM_PRO_TASSE

    def entnehmen(self, tassen: int) -> bool:
        if tassen <= 0 or tassen > self.tassen:
            return False
        self._fuellstand_g -= tassen * GRAMM_PRO_TASSE
        return True


b = Bohnenbehaelter(250)
b.fuellstand_g = 300
b.fuellstand_g = 100
print(b.fuellstand_g, b.tassen)
print(b.entnehmen(13), b.entnehmen(3), b.fuellstand_g, b.tassen)
assert b.fuellstand_g == 76 and b.tassen == 9
assert not b.entnehmen(0)
assert not b.entnehmen(10)
assert b.entnehmen(9) and b.fuellstand_g == 4 and b.tassen == 0
b.fuellstand_g = -5
```

```
assert b.fuellstand_g == 4
assert b.kapazitaet_g == 250
```

```
Ungültiger Füllstand: 300 g
100 12
False True 76 9
Ungültiger Füllstand: -5 g
```

100 g reichen für 100 // 8 = 12 Tassen. 13 Tassen gehen nicht, 3 Tassen schon: 100 − 24 = 76 g, das sind 9 Tassen.

Bewertung: 1 Konstante mit `Final`, 1 `__init__` mit `_`-Attributen, 1 Property `kapazitaet_g` ohne Setter, 2 Property `fuellstand_g` mit Setter und Prüfung beider Grenzen, 1 berechnete Property mit `//`, 2 `entnehmen()` mit beiden Fällen und `bool`-Rückgabe. Ob der Setter `print` + `return` nutzt oder `raise ValueError(…)`, ist laut Notizen noch offen. Beides ist hier richtig.

b)

```
Bruehung(sorte='Espresso', tassen=2, stark=False)
True False
True
```

`@dataclass` erzeugt `__init__`, `__repr__` und `__eq__`. `==` vergleicht deshalb die Attribute (gleich), `is` prüft, ob es dasselbe Objekt ist (nein, zwei Objekte). Bei einer normalen Klasse ohne `__eq__` wäre auch `erste == zweite` falsch.

Bewertung: je Zeile 1 Punkt.

c)

1. `b.tassen = 10`: ty meldet einen Fehler, beim Ausführen bricht das Programm ab.

```
error[invalid-assignment]: Cannot assign to read-only property `tassen` on objec
```

```
AttributeError: property 'tassen' of 'Bohnenbehaelter' object has no setter
```

2. `b.fuellstand_g = 500`: ty meldet **nichts**, denn 500 ist ein `int`. Ob der Wert in den Behälter passt, prüft erst der Setter beim Ausführen. Ausgabe: `Ungültiger Füllstand:`

`500 g`, der Füllstand bleibt 0.

Bewertung: 1 Punkt ty-Meldung bei (1), 1 Punkt `AttributeError`, 1 Punkt für (2) mit „ty meldet nichts“.

Aufgabe 7

a)

```
Wasser (150 ct)
Tee (220 ct, 80 °C)
Süß: Kakao (270 ct, 65 °C)
True Kakao
False 270
```

1. `Heissgetraenk.__init__` gibt `preis + 20` an die Oberklasse weiter: Tee kostet 220, Kakao 270.
2. `Kakao` hat kein eigenes `__init__` und erbt das von `Heissgetraenk`.
3. `beschreibung()` steht in `Getraenk` und ruft `self.preis_text()` auf. Welche Fassung läuft, entscheidet das **Objekt**: beim Tee die überschriebene aus `Heissgetraenk`. Das ist Polymorphie.
4. Beim Kakao: `Kakao.beschreibung()` → `super().beschreibung()` findet `Getraenk.beschreibung()` → `self.preis_text()` ist wieder `Heissgetraenk.preis_text()` → `super().preis_text()` ist `Getraenk.preis_text()`.
5. Ein Kakao ist auch ein Heißgetränk (`isinstance` gilt für Unterklassen). Ein Tee ist kein Kakao. `type()` nennt die genaue Klasse.

Bewertung: Zeile 1 und 2 je 1 Punkt, Zeile 3 2 Punkte (1 für „Süß: “, 1 für den Rest), Zeile 4 und 5 je 1 Punkt.

b)

```
from abc import ABC, abstractmethod
from decimal import Decimal
from typing import override


class Versandart(ABC):
    def __init__(self, name: str) -> None:
        self.name = name

    @abstractmethod
    def kosten(self, gewicht_g: int) -> Decimal:
        """Versandkosten in Euro für ein Paket mit diesem Gewicht."""

    def angebot(self, gewicht_g: int) -> str:
        return f"{self.name}: {self.kosten(gewicht_g)} Euro"


class Standard(Versandart):
    def __init__(self, name: str = "Standard") -> None:
        super().__init__(name)

    @override
    def kosten(self, gewicht_g: int) -> Decimal:
        if gewicht_g <= 2000:
            return Decimal("4.99")
        return Decimal("6.99")


class Express(Standard):
    def __init__(self) -> None:
        super().__init__("Express")

    @override
    def kosten(self, gewicht_g: int) -> Decimal:
        return super().kosten(gewicht_g) + Decimal("5.00")


class Abholung(Versandart):
    def __init__(self, filiale: str) -> None:
        super().__init__(f"Abholung in {filiale}")

    @override
    def kosten(self, gewicht_g: int) -> Decimal:
        return Decimal("0.00")


def guenstigste(arten: list[Versandart], gewicht_g: int) -> Versandart:
    beste = arten[0]
```

```
    for art in arten:
        if art.kosten(gewicht_g) < beste.kosten(gewicht_g):
            beste = art
    return beste


arten: list[Versandart] = [Express(), Standard(), Abholung("Mitte")]
for art in arten:
    print(art.angebot(2500))
print("Am günstigsten:", guenstigste(arten, 2500).name)

assert Standard().kosten(2000) == Decimal("4.99")
assert Standard().kosten(2001) == Decimal("6.99")
assert Express().kosten(500) == Decimal("9.99")
assert Express().name == "Express"
assert guenstigste([Express(), Standard()], 100).name == "Standard"
assert guenstigste([Standard(), Express()], 100).name == "Standard"
print("alle Tests bestanden")
```

```
Express: 11.99 Euro
Standard: 6.99 Euro
Abholung in Mitte: 0.00 Euro
Am günstigsten: Abholung in Mitte
alle Tests bestanden
```

`ty check` meldet für diese Lösung nichts. Wichtig: `Decimal("4.99")` mit Anführungszeichen. `Decimal(4.99)` übernimmt die Ungenauigkeit des `float`.

Bewertung (10 Punkte):

- 2: `Versandart(ABC)` mit `@abstractmethod kosten` und konkreter Methode `angebot()`, die `self.kosten()` aufruft
- 2: `Standard` mit beiden Gewichtsfällen (Grenze 2000 g einschließlich)
- 2: `Express(Standard)` mit `super().kosten(…)` statt neu geschriebener Preise
- 1: `Abholung` mit Name über `super().__init__(…)`
- 2: `guenstigste()` ohne `isinstance`, ruft nur `kosten()` auf
- 1: Typangaben, `Decimal` aus Text, `@override`

c) `Start` wird noch ausgegeben. Erst bei `Brief()`, also beim Erzeugen des Objekts, bricht Python ab. Die Klassendefinition selbst ist noch kein Fehler.

```
Start
TypeError: Can't instantiate abstract class Brief without an implementation for
```

ty findet das schon vor dem Ausführen:

```
error[call-non-callable]: Cannot instantiate abstract class `Brief`
```

`porto()` hilft nicht: Für Python ist das eine ganz andere Methode.

Bewertung: 1 Punkt `TypeError` beim Erzeugen (nicht bei `class Brief`), 1 Punkt „ty meldet es vorher“.

Aufgabe 8

a) Die echten Meldungen von `ty check`:

Stelle	ty meldet	Grund
`def groesse(…) -> str`	`invalid-return-type`	„Function can implicitly return None“: Für `menge <= 0` gibt es kein `return`
(1) `anzahl = "drei"`	`invalid-assignment`	Ein `str` passt nicht zur Typangabe `int`
(2) `MWST = 0.07`	`invalid-assignment`	„Reassignment of Final symbol MWST is not allowed“
(3) `groesse(0)`	nichts	Der Aufruf selbst ist korrekt, der Fehler steht in der Funktion
(4) `brutto(10)`	nichts	Ein `int` darf überall stehen, wo `float` erwartet wird
(5) `brutto("10")`	`invalid-argument-type`	„Expected float, found Literal["10"]“

Bewertung: je Zeile (1) bis (5) 1 Punkt. Bei (3) gibt es den Punkt nur, wenn klar ist, dass ty die **Funktion** bemängelt und nicht den Aufruf.

b) Python ignoriert Typangaben und `Final`. Die Zeilen (1) und (2) laufen ohne Fehler, `MWST` ist danach 0.07.

```
None
10.7
Traceback (most recent call last):
  …
TypeError: can't multiply sequence by non-int of type 'float'
```

- (3): `groesse(0)` erreicht kein `return`, deshalb `None`.
- (4): 10 · (1 + 0.07) gerundet ist 10.7, nicht 11.9. Die „Konstante“ wurde in (2) wirklich geändert.
- (5): `"10" * 1.07` geht nicht. Das Programm bricht ab, `fertig` wird nicht mehr ausgegeben.

Bewertung: je Punkt 1 Punkt (`None`; 10.7; Abbruch mit `TypeError` ohne „fertig“).

c)

```
def groesse(menge: int) -> str:
    if menge > 10:
        return "groß"
    elif menge > 0:
        return "klein"
    return "keine"


assert groesse(0) == "keine"
assert groesse(11) == "groß"
assert groesse(10) == "klein"
print("alle Tests bestanden")
```

```
alle Tests bestanden
```

Statt des letzten `return` geht auch `else: return "keine"`. Danach meldet ty für `groesse()` nichts mehr.

Bewertung: 1 Punkt für einen Rückgabewert in jedem Zweig, 1 Punkt für `"keine"` bei 0.

Aufgabe 9

a) und **b)** Die beiden neuen Methoden. Sie stehen in den Klassen `Lager` und `Warenkorb` aus der Aufgabe:

```
    def nachbestellen(self, mindestbestand: int) -> list[Produkt]:
        knapp: list[Produkt] = []
        for produkt in self.produkte:
            if produkt.bestand < mindestbestand:
                knapp.append(produkt)
        return knapp
```

```
    def reduzieren(self, nummer: int, menge: int) -> bool:
        position = self.position_zu(nummer)
        if position is None or menge < 1:
            return False
        position.menge -= menge
        if position.menge <= 0:
            self.positionen.remove(position)
        return True
```

Getestet mit den vollständigen Klassen:

```
lager = Lager([
    Produkt(101, "Kaffee 500 g", Decimal("8.90"), 12),
    Produkt(102, "Espresso 250 g", Decimal("5.40"), 3),
    Produkt(204, "Thermobecher", Decimal("14.50"), 0),
])
for produkt in lager.nachbestellen(5):
    print(produkt.name)
assert len(lager.nachbestellen(5)) == 2
assert lager.nachbestellen(0) == []

korb = Warenkorb()
kaffee = lager.finden(101)
espresso = lager.finden(102)
assert kaffee is not None and espresso is not None
korb.hinzufuegen(kaffee, 4)
korb.hinzufuegen(espresso, 2)
assert korb.reduzieren(101, 1) and korb.positionen[0].menge == 3
assert korb.reduzieren(102, 5) and len(korb.positionen) == 1
assert not korb.reduzieren(102, 1)
assert not korb.reduzieren(101, 0)
assert korb.reduzieren(101, 3) and korb.positionen == []
print("alle Tests bestanden")
```

```
Espresso 250 g
Thermobecher
alle Tests bestanden
```

Bewertung a) (3 Punkte): 1 leere Liste vor der Schleife, 1 Vergleich mit `<`, 1 `append` und `return` nach der Schleife.

Bewertung b) (5 Punkte): 1 Position mit `position_zu()` suchen, 1 `False` bei fehlender Position oder ungültiger Menge, 1 Menge verringern, 1 Position bei 0 oder weniger entfernen, 1 `True` am Ende. Wer `remove()` vor der Prüfung auf `None` aufruft, verliert einen Punkt: `remove(None)` gibt einen `ValueError`.

c)

```
True
False
True
55.70
2 0 0 2
False
```

Aufruf	im_korb	Prüfung	Ergebnis
hinzufuegen(kaffee, 3)	0	0 + 3 > 5 nein	True
hinzufuegen(lager.finden(101), 3)	3	3 + 3 > 5 ja	False
hinzufuegen(becher, 2)	0	0 + 2 > 2 nein	True

1. `lager.finden(101)` liefert **dasselbe** Objekt wie `kaffee`. Deshalb zählen die 3 Stück im Korb mit.
2. `bestellen()`: 3 · 8.90 + 2 · 14.50 = 26.70 + 29.00 = 55.70. Die Bestände sinken auf 5 − 3 = 2 und 2 − 2 = 0. `kaffee` und `becher` sind dieselben Objekte wie in den Positionen, deshalb sieht man die neuen Bestände auch über diese Namen.
3. `self.positionen = []` gibt dem Korb eine **neue** Liste. `gemerkt` zeigt weiter auf die alte Liste mit 2 Positionen.
4. Der Becher hat jetzt Bestand 0: 0 + 1 > 0, also `False`.

ty meldet die Zeile `korb.hinzufuegen(lager.finden(101), 3)`:

```
error[invalid-argument-type]: Argument to bound method `Warenkorb.hinzufuegen` i
  Expected `Produkt`, found `Produkt | None`
```

`finden()` kann `None` liefern, `hinzufuegen()` erwartet aber ein `Produkt`. Richtig ist, vorher mit `if produkt is not None:` oder `assert produkt is not None` zu prüfen.

Bewertung: 1 Punkt für die drei `True` / `False`-Zeilen, 1 Punkt für `55.70`, 1 Punkt für `2 0 0 2` und das letzte `False`, 1 Punkt für die ty-Zeile mit Begründung.

Auswertung

Zähl deine Punkte zusammen. Die Grenzen sind eigene Richtwerte, keine offizielle Notenskala.

Punkte	Einschätzung	Was du jetzt tust
108–120	Sehr sicher	Halte das Niveau. Wiederhole kurz vor der Klausur die Fallen-Abschnitte („Achtung“) aus Kapitel 3 bis 7.
90–107	Gut vorbereitet	Schau dir die Aufgaben an, in denen du Punkte verloren hast, und rechne die passenden Übungen der Kapitel noch einmal.
60–89	Bestanden, aber mit Lücken	Nimm dir die zwei Aufgaben mit den wenigsten Punkten vor und arbeite die passenden Kapitel (Tabelle unten) ganz durch. Schreib danach die Aufgabe neu, ohne in die Lösung zu schauen.
0–59	Noch nicht bestanden	Fang bei den Grundlagen an: Kapitel 2 (Rundentabellen), dann Kapitel 4 und 5. Erst danach Kapitel 6 und 7. Mach die Probeklausur in einer Woche noch einmal.

Welche Kapitel zu welcher Aufgabe gehören:

Wenig Punkte in	Wiederhole
Aufgabe 1 (Schleifen)	Kapitel 2 „for-Schleifen ganz langsam“, Kapitel 4 (while, break, continue)
Aufgabe 2 (Namen, Gültigkeit, Operatoren)	Kapitel 3 (Operatoren, Datentypen), Kapitel 4 (Listen), Kapitel 5 (Gültigkeitsbereiche und Parameterübergabe)
Aufgabe 3 (Rekursion)	Kapitel 5, Abschnitt Rekursion
Aufgabe 4 (Fehlersuche)	Kapitel 3 (Fehlerarten), Kapitel 4 (for mit range), Kapitel 5 (Rückgabewerte)
Aufgabe 5 (Funktionen schreiben)	Kapitel 5, dazu die Muster „aufsummieren“ und „zählen“ aus Kapitel 2
Aufgabe 6 (Properties, dataclass)	Kapitel 6, Abschnitte Kapselung mit Properties und @dataclass
Aufgabe 7 (Vererbung, ABC)	Kapitel 7, alle Abschnitte
Aufgabe 8 (ty)	Kapitel 3 (Konstanten mit Final), Kapitel 5 (Rückgabewerte), Kapitel 7 (Typangabe vs. Objekt)
Aufgabe 9 (Kaffeeshop)	Kapitel 8, Stufe 1 (Lager, Warenkorb), Kapitel 6 (Namen und Objekte)